

Choosing a transformation engine

Glue, EMR, Lambda, Redshift, Athena, and where large language models now fit.

Ask what the work **is** before asking which service does it. The work picks the engine.

The engines, in one sentence each

ENGINE	WHAT IT IS FOR
AWS Glue	Serverless Spark. Scheduled ETL with nothing to operate
Amazon EMR	The whole open-source ecosystem, with control and tuning
AWS Lambda	Small, short, per-record or per-file work
Amazon Redshift	SQL over data already in the warehouse
Amazon Athena	SQL over data already in the lake, occasionally
Amazon Bedrock	Meaning, not arithmetic. Unstructured input

ON THE EXAM

The last two rows are the ones candidates forget. Not every transformation needs a compute cluster.

Volume, velocity and variety

- ◆ **Volume.** How much. Decides whether the work must be distributed at all.
- ◆ **Velocity.** How fast it arrives and how fast an answer is needed.
- ◆ **Variety.** Structured, semi-structured, or unstructured.
- ◆ **Variety is the one that has changed.** Unstructured now has a managed answer.
- ◆ Volume rarely decides anything alone. Every engine here handles large volumes.

ON THE EXAM

The exam guide names these three explicitly. Treat them as three separate questions to ask, not a slogan.

Transform before loading, or after?

ETL

Transform, then load

- Glue or EMR reshapes data on the way
- Only clean data reaches the target
- Warehouse compute stays for querying
- The classic pipeline shape

ELT

Load, then transform

- Raw data lands, SQL reshapes it in place
- Uses the warehouse engine you already pay for
- No separate cluster or job to operate
- Needs a target that can do the work

THE TIEBREAKER

If data is already in Redshift, transforming it with SQL beats exporting it, reshaping it, and loading it back.

EMR exists for the cases where **Glue is not enough**. Knowing what those are is the exam point.

The three reasons to choose EMR

- ◆ **An engine Glue does not run.** Hive, HBase, Presto, Trino, Flink, Pig.
- ◆ **An existing Hadoop estate** being migrated, where rewriting costs more than lifting.
- ◆ **Deep tuning or custom software** installed alongside the framework.
- ◆ **Plus one economic reason.** Spot Instances on a sustained workload.
- ◆ Absent all four, Glue wins on operational overhead.

ON THE EXAM

If the question names Hive, HBase, Presto or Trino, Glue is eliminated before you read further.

The three ways to run EMR

OPTION	CHOOSE IT WHEN
EMR on EC2	You want cluster control, Spot Instances and custom software
EMR Serverless	Short or unpredictable jobs. No cluster, per-second billing
EMR on EKS	You already run Kubernetes and want to share that capacity

ON THE EXAM

EMR Serverless removes most of the operational-overhead argument against EMR. Do not assume EMR means a cluster.

Cluster anatomy, for EMR on EC2

- ◆ **Primary node.** Coordinates. If it goes, the cluster goes.
- ◆ **Core nodes.** Run tasks and hold HDFS. Losing one risks data.
- ◆ **Task nodes.** Run tasks only. No HDFS. Safe to lose.
- ◆ **EMRFS** reads and writes Amazon S3 directly, so HDFS is often scratch space.
- ◆ HDFS lives on the cluster. Terminate the cluster and it is gone.

ON THE EXAM

Storing data in S3 rather than HDFS is what makes a transient cluster possible at all.

Making EMR cheap

- ◆ **Spot Instances on task nodes.** Up to ninety percent off, and interruption is survivable.
- ◆ **Keep primary and core on-demand.** Losing them costs you the cluster or the data.
- ◆ **Instance fleets** mix several instance types, so one Spot pool drying up is not fatal.
- ◆ **Managed scaling** resizes the cluster to the workload between limits you set.
- ◆ **Transient clusters** for scheduled work. Nothing runs between jobs.

ON THE EXAM

Instance groups are one instance type each. Instance fleets mix types and are the answer to Spot availability questions.

A media company runs an Amazon EMR cluster on EC2 twenty four hours a day. The cluster runs a nightly Apache Hive job that takes about three hours, and nothing else. It uses fifteen on-demand core nodes and stores intermediate data in HDFS on those nodes.

Finance has flagged the cost. The job must still complete overnight, and the team has a large body of existing Hive scripts they do not want to rewrite.

Which combination will reduce cost the most while meeting the requirements?

- ☐ A Migrate the Hive scripts to AWS Glue and run them as scheduled Spark jobs
- ☐ B Store data in Amazon S3 with EMRFS, run a transient cluster nightly, and use Spot Instances for task nodes
- ☐ C Switch all fifteen core nodes to Spot Instances and keep the cluster running
- ☐ D Reduce the cluster from fifteen nodes to five and accept a longer runtime

◆ PAUSE AND DECIDE

Which combination will reduce cost the most while meeting the requirements?

- A

 Migrate Hive to Glue
Glue does not run Hive, so this means rewriting every script in Spark. Explicitly ruled out by the question.
- B

 S3 with EMRFS, transient cluster, Spot task nodes
Stops paying for twenty one idle hours a day, and discounts the three that remain. Hive still runs unchanged.
- C

 All core nodes on Spot
Core nodes hold HDFS. An interruption loses data mid-job, and the cluster still runs all day regardless.
- D

 Five nodes, longer runtime
Saves two thirds of a bill you should not be paying at all. The waste is the idle time, not the node count.

THE SIGNAL

Twenty four hours of cluster for three hours of work. The waste is time, not size.

Containers for data workloads

- ◆ **Amazon ECS** for containerised jobs with no Kubernetes in the organisation.
- ◆ **Amazon EKS** when Kubernetes is already the platform standard.
- ◆ **AWS Fargate** removes the instances under either of them.
- ◆ **EMR on EKS** runs Spark in pods on a cluster you already have.
- ◆ Containers rarely beat Glue for plain ETL. They win on portability and existing standards.

ON THE EXAM

The exam guide names optimising container usage as a skill. It usually appears as an existing-platform argument.

Where the development happens

- ◆ **EMR Studio.** Notebooks against EMR clusters, with Git integration.
- ◆ **Glue interactive sessions.** Jupyter against live Glue compute, billed per second.
- ◆ **Athena notebooks** run Apache Spark against the lake with no cluster.
- ◆ **Explore interactively, then schedule** the finished code as a job.
- ◆ **Interactive sessions bill while open.** An idle notebook is a running cost.

ON THE EXAM

A notebook left open overnight is a real and frequently examined source of cost.

A company processes a single compressed CSV file that lands in Amazon S3 once an hour. The file is between four and nine megabytes. Processing means renaming six columns, casting two date fields, and writing the result as Parquet.

The current implementation is an AWS Glue Spark job triggered by an S3 event. The finance team has asked why a job that handles nine megabytes costs what it does.

What should the data engineer change?

- ☐ A Reduce the Glue job from ten DPUs to the minimum of two
- ☐ B Replace the Glue Spark job with an AWS Lambda function triggered by the same event
- ☐ C Replace the Glue Spark job with a Glue Python shell job
- ☐ D Batch the files and run one Glue job daily instead of hourly

◆ PAUSE AND DECIDE

What should the data engineer change?

- A

 Drop to two DPUs
Real improvement, and still a Spark cluster starting up for nine megabytes, with a one minute billing floor each time.
- B

 Lambda on the same event
Megabytes, three trivial transformations, well inside fifteen minutes. Milliseconds to start and no minimum billing floor.
- C

 Glue Python shell job
Genuinely close. Cheaper than Spark, still a Glue job with startup time and a one minute minimum. Lambda is cheaper again.
- D

 Batch to daily
Cuts cost by delaying every file by up to a day. Nothing suggested that latency was negotiable.

THE SIGNAL

Compare with scenario 1. There EMR was right and misconfigured. Here Spark is the wrong category of engine entirely.

The cheapest pipeline is often the one you
do not build.

Redshift as the engine

- ◆ **CREATE TABLE AS SELECT** materialises a query result as a new table.
- ◆ **Stored procedures** hold multi-step logic inside the warehouse.
- ◆ **Materialized views** precompute and refresh, incrementally where possible.
- ◆ **Redshift Spectrum** joins warehouse tables to data still sitting in S3.
- ◆ **Python UDFs end on the thirtieth of June, twenty twenty six.** Use SQL or Lambda UDFs.

ON THE EXAM

Spectrum means "the data is in S3" does not force you out of Redshift. You can join across both.

Athena as the engine

- ◆ **CREATE TABLE AS SELECT** converts format, partitions and compacts in one statement.
- ◆ **INSERT INTO** adds to an existing table, so CTAS plus inserts builds it up.
- ◆ **One hundred partitions maximum** per CTAS or INSERT INTO statement.
- ◆ **No stored procedures.** Multi-step logic needs orchestrating outside Athena.
- ◆ **MERGE** works, but only on transactional table formats like Iceberg.

ON THE EXAM

The hundred-partition ceiling is a real exam detail. Beyond it you chain INSERT INTO statements or use Glue.

Athena CTAS or a Glue job?

ATHENA CTAS

SQL, once or occasionally

- No job, no cluster, no schedule
- Billed per query by bytes scanned
- Under one hundred partitions per statement
- Logic must fit in SQL

GLUE JOB

Code, repeatedly

- Arbitrary transformation logic
- Bookmarks for incremental processing
- No partition ceiling
- A job to schedule and monitor

THE TIEBREAKER

A one-off conversion is CTAS. A nightly incremental pipeline is Glue.

An analytics team has three years of application logs in Amazon S3 as gzipped JSON, in a single prefix with no partitioning, totalling about two terabytes. Analysts query them with Amazon Athena and every query scans the whole dataset.

The team wants a one-off conversion to Parquet partitioned by day. There are roughly eleven hundred days of data. After the conversion, a new daily file will continue to arrive and must be appended in the same format.

What is the most appropriate approach?

- ☐ A Run a single Athena CTAS statement partitioned by day to create the whole table
- ☐ B Run an AWS Glue job for the historical conversion, then a scheduled Glue job with bookmarks for the daily file
- ☐ C Run an Athena CTAS statement for the first hundred days, then INSERT INTO for the rest, and a daily INSERT INTO afterwards
- ☐ D Enable S3 Intelligent-Tiering on the prefix to reduce the cost of scanning

◆ PAUSE AND DECIDE

What is the most appropriate approach?

- A

 One CTAS for the whole table
Eleven hundred partitions against a hundred-partition ceiling. Fails with too many open writers.
- B

 Glue for history, scheduled Glue with bookmarks for the daily file
No partition ceiling on the backfill, and bookmarks handle the ongoing file. Both halves of the requirement, one tool.
- C

 CTAS then a chain of INSERT INTO
Works, and needs eleven statements hand-run for the backfill plus a scheduled daily one. More moving parts than B.
- D

 Intelligent-Tiering
Changes storage cost. Athena charges for bytes scanned, which is unaffected by storage class.

THE SIGNAL

Eleven hundred partitions is the planted number. It exists to make you check the CTAS limit.

Four ways to convert a format

APPROACH	BEST WHEN
Amazon Data Firehose	Streaming data. Convert on arrival, no job at all
Athena CTAS	A one-off or occasional conversion, under 100 partitions
AWS Glue job	Recurring, incremental, or with real transformation logic
Amazon EMR	Very large volumes, or a non-Spark engine is already in use

ON THE EXAM

The first row is the one people miss. If data is still arriving on a stream, convert it before it lands.

Joining data from several sources

- ◆ **Land everything first.** Joining in the lake beats joining across live systems.
- ◆ **Redshift Spectrum** joins warehouse tables to S3 without moving either.
- ◆ **Athena federated query** reaches into other data sources through connectors.
- ◆ **Schema conflicts** are the real work. Same field, two types, two names.
- ◆ **Glue DynamicFrames** with `resolveChoice` handle conflicting types without failing.

ON THE EXAM

Federated query is convenient and puts load on the source system. Landing the data first usually wins.

A retailer needs a daily report joining order data in Amazon Redshift, product reference data in Amazon RDS for PostgreSQL, and clickstream files in Amazon S3. The RDS database also serves the live checkout application.

A data engineer proposes using Amazon Athena federated query to join all three sources at report time. The reporting query is heavy and runs each morning at eight.

What is the main problem, and what should be done instead?

- ☐ A Athena cannot query Redshift. Move all data into Redshift and query it there
- ☐ B The federated query loads the live checkout database. Replicate the RDS reference data into the lake or warehouse first
- ☐ C Federated query is too slow. Increase the Athena workgroup concurrency limit
- ☐ D There is no problem. Federated query is designed for exactly this pattern

◆ PAUSE AND DECIDE

What is the main problem, and what should be done instead?

- A

Athena cannot query Redshift

It can, through a connector. The premise is false, which is enough to eliminate it.
- B

Replicate the RDS data first

A heavy analytical join runs directly against the checkout database. Replicating with DMS or zero-ETL isolates the two.
- C

Raise the concurrency limit

Concurrency governs how many queries run at once. One heavy query is not a concurrency problem.
- D

No problem

Federated query is real and appropriate for occasional lookups. A heavy daily report against a live transactional system is not that.

THE SIGNAL

"Also serves the live checkout application" is the whole question. It turns an elegant design into an outage risk.

Why some Spark steps are cheap and others are not

NARROW TRANSFORMATION

Each partition alone

- filter, map, select, withColumn
- No data moves between executors
- Chains into a single stage
- Effectively free to add

WIDE TRANSFORMATION

Data must be redistributed

- groupBy, join, distinct, repartition
- A shuffle crosses the network
- Starts a new stage
- Where the time and failures live

THE TIEBREAKER

Every performance problem in this module happens at a shuffle. Skew, memory pressure and slow stages are all shuffle problems.

Reading the same data twice

- ◆ **Spark is lazy.** Nothing runs until an action asks for a result.
- ◆ **Two actions on one DataFrame** recompute the whole lineage twice.
- ◆ **Cache or persist** keeps the intermediate result for reuse.
- ◆ **Broadcast a small table** so a join needs no shuffle at all.
- ◆ **Caching costs memory.** Caching everything causes the spills it was meant to prevent.

ON THE EXAM

A job that reuses an expensive intermediate result several times, without caching, does that work several times.

Why transformations fail

SYMPTOM	CAUSE
Out of memory on executors	Partition too large, or a wide transformation. Bigger workers or more partitions
Out of memory on the driver	Collecting results to the driver, or a very large query plan
One task runs far longer than the rest	Data skew on a key. Repartition, salt, or use a memory-optimised worker
Job slow before processing starts	Millions of small files to list and open
Reads far more than it needs	No pushdown predicate or partition filter
Fails only on some runs	A schema change in the source, or a type conflict

ON THE EXAM

The last row is the one people miss. A job that worked yesterday and fails today usually met new data, not new code.

Skew, and why capacity does not fix it

- ◆ **Skew** is one key holding far more rows than the others.
- ◆ **Adding workers changes nothing.** They idle while one task grinds on.
- ◆ **Salting** adds a random suffix to spread a hot key across partitions.
- ◆ **Broadcast the small side** of a join so no shuffle happens at all.
- ◆ **R-type Glue workers** give more memory per worker for skewed aggregations.

ON THE EXAM

Skew is the failure mode where the intuitive fix provably does not work. That is why the exam likes it.

An AWS Glue job joins a transactions table to a customer table and aggregates by customer. It runs on twenty G.2X workers. Ninety-five percent of tasks finish within four minutes; two tasks run for over an hour, and the job's total runtime is dominated by them.

Investigation shows that two customer identifiers account for a large share of all transactions, because they are internal test accounts. The customer table itself is small, around forty megabytes.

Which change will most improve the runtime?

- ☐ A Increase the worker count from twenty to sixty
- ☐ B Filter out the two internal test accounts, and broadcast the small customer table in the join
- ☐ C Change the worker type from G.2X to G.8X
- ☐ D Enable Glue Auto Scaling with a maximum of sixty workers

◆ PAUSE AND DECIDE

Which change will most improve the runtime?

- A

 Triple the workers
Forty extra executors idle while the same two tasks run. Skew is immune to capacity.
- B

 Filter the test accounts and broadcast the small table
Removes the skewed keys entirely, and a forty megabyte table broadcasts to every executor so the join needs no shuffle.
- C

 Move to G.8X workers
More memory per worker helps a skewed task survive. It does not make one task stop being most of the work.
- D

 Auto Scaling to sixty
Adds workers when there is pending work to parallelise. Two long tasks offer nothing to parallelise.

THE SIGNAL

Ninety-five percent finishing in four minutes while two run for an hour is the textbook description of skew.

Making transformed data available

- ◆ **API Gateway with Lambda** for a custom API over any store.
- ◆ **Redshift Data API** runs SQL over HTTPS with no persistent connection or driver.
- ◆ **Athena API** starts a query and collects results asynchronously.
- ◆ **The Data API suits Lambda**, which cannot hold a connection pool between invocations.
- ◆ **AWS AppSync** when consumers want GraphQL rather than REST.

ON THE EXAM

The Redshift Data API exists because serverless functions and traditional database connections do not mix.

Maintaining a data API once it exists

- ◆ **Throttling and usage plans** stop one consumer exhausting the backend.
- ◆ **Caching responses** removes repeated identical queries entirely.
- ◆ **Versioning** lets the schema change without breaking existing consumers.
- ◆ **Pagination** so one request cannot ask for an unbounded result set.
- ◆ **Authorisation per consumer**, so teams see only what they should.

ON THE EXAM

The exam guide names consuming and maintaining data APIs, not only creating them. Throttling appears most.

A company exposes an internal API that lets other teams request aggregated sales figures. The API is backed by AWS Lambda functions that query Amazon Redshift using a JDBC connection opened at the start of each invocation.

Under load, teams report intermittent failures and slow responses. CloudWatch shows Lambda invocations timing out while waiting to connect, and Redshift reporting that the connection limit has been reached.

What should the data engineer change?

- ☐ A Increase the Lambda function timeout and add retries with exponential backoff
- ☐ B Replace the JDBC connections with calls to the Amazon Redshift Data API
- ☐ C Increase the Redshift cluster node count to raise the connection limit
- ☐ D Move the API from Lambda to containers on Amazon ECS so connections can be pooled

◆ PAUSE AND DECIDE

What should the data engineer change?

- A

 Longer timeout and retries
Makes each failure take longer and adds more connection attempts to a system already out of connections.
- B

 Use the Redshift Data API
No persistent connection at all. Statements go over HTTPS, so concurrent Lambda invocations stop competing for connection slots.
- C

 Add Redshift nodes
Buys headroom by scaling the warehouse for a connection problem. Expensive, and the pattern still does not fit.
- D

 Move to ECS for pooling
Would genuinely work, by abandoning serverless to accommodate a driver. A managed API removes the need.

THE SIGNAL

Compare with scenario 5. There the reflex was more capacity; here it is more time. Both avoid the actual mismatch.

Some transformations are not calculations.
They are **judgements about meaning**.

What LLMs do in a pipeline

- ◆ **Classification.** Route a support ticket, tag a document, assign a category.
- ◆ **Extraction.** Pull named fields out of unstructured text or a scanned form.
- ◆ **Summarisation.** Condense long text into something a warehouse column can hold.
- ◆ **Enrichment.** Add sentiment, topic or intent to each record.
- ◆ **Structuring.** Turn documents, images, audio or video into rows and columns.

ON THE EXAM

Every one of these turns unstructured input into something a normal analytics pipeline can handle.

Which service for which job

SERVICE	USE IT FOR
Amazon Textract	Reading text, forms and tables out of scanned documents
Amazon Comprehend	Entities, sentiment, language and PII detection in text
Amazon Bedrock	General purpose. Prompt a model for classification or extraction
Bedrock Data Automation	Documents, images, audio and video to structured output, one API
Bedrock Knowledge Bases	Retrieval over your own content, structured or unstructured
Amazon Q and Kendra	Natural language search and assistance over enterprise content

ON THE EXAM

Bedrock Data Automation is the managed answer for intelligent document processing. It replaces stitching three services together.

Purpose-built or generative?

TEXTTRACT AND COMPREHEND

Fixed capabilities

- Trained for specific tasks
- Deterministic and repeatable
- Cheaper per document
- Cannot go beyond their categories

BEDROCK

Prompted capabilities

- Any extraction you can describe
- Handles formats it has not seen
- Costs more per document
- Output varies between runs

THE TIEBREAKER

A fixed set of entities is Comprehend. An arbitrary schema described in a prompt is Bedrock.

The economics of running a model over data

- ◆ **Batch inference** is roughly half the price of on-demand invocation.
- ◆ It needs at least one hundred requests, and runs when capacity allows.
- ◆ Latency is minutes to hours, not seconds. Wrong for anything interactive.
- ◆ Right for pipelines, where nobody is waiting on an individual record.
- ◆ Cost scales per document, so filter before you invoke, not after.

ON THE EXAM

A nightly pipeline classifying yesterday's tickets is the exact shape batch inference exists for.

Being honest about the limits

- ◆ **Not deterministic.** The same input can produce different output.
- ◆ **It can be confidently wrong.** Extraction needs validation, not trust.
- ◆ **Expensive per record** compared to any conventional transformation.
- ◆ **Sending data to a model** is a data governance decision, not just a technical one.
- ◆ **Use it where nothing else can work.** Not where SQL would do.

HONEST LIMITS

An LLM is the wrong answer to a question a regular expression could answer. The exam does test that.

An insurance company receives around twelve thousand scanned claim documents each night as PDFs. Layouts differ by broker. The company needs six fields from each document, including the policy number, incident date and claim amount, loaded into Amazon Redshift by eight in the morning.

A previous attempt using a rules-based parser reached about sixty percent accuracy because the layouts vary too much. Results are needed by morning; nobody is waiting on an individual document.

Which approach best meets the requirements?

- | | |
|---|---|
| <p>A An AWS Glue job with regular expressions tuned for each broker's layout</p> | <p>B Amazon Bedrock Data Automation in a nightly pipeline, using batch processing, loading results into Redshift</p> |
| <p>C Amazon Comprehend to detect entities in the extracted text</p> | <p>D Real-time Amazon Bedrock invocation per document as each one arrives</p> |

◆ PAUSE AND DECIDE

Which approach best meets the requirements?

- A

 Glue with per-broker regular expressions
This is the approach that already reached sixty percent, scaled up. Varying layouts are what defeat rules.
- B

 Bedrock Data Automation, nightly batch
Handles varying layouts, extracts an arbitrary field schema, and nobody waits per document so batch pricing applies.
- C

 Comprehend entity detection
Finds predefined entity types. Policy number and claim amount are business-specific fields, not standard entities.
- D

 Real-time Bedrock per document
Right model, wrong economics. Interactive pricing for work with an overnight deadline and no waiting user.

THE SIGNAL

Three details decide it: layouts vary, the fields are business-specific, and nobody is waiting.

Cost levers, by engine

- ◆ **Glue.** Right-size DPUs, use Flex, enable bookmarks, push down predicates.
- ◆ **EMR.** Spot on task nodes, instance fleets, transient clusters, managed scaling.
- ◆ **Lambda.** Tune memory, because memory buys CPU and often shortens billed time.
- ◆ **Athena.** Partition and compress. You are billed for bytes scanned.
- ◆ **Bedrock.** Batch inference, and filter the dataset before invoking.

ON THE EXAM

Every one of these is about doing less work, not about buying a cheaper unit of the same work.

A data platform team runs four workloads. A nightly Spark ETL job with no deadline. An hourly conversion of one small file. A daily report joining three curated Redshift tables. A monthly classification of twenty thousand free-text survey responses into six themes.

The team has been asked to reduce cost across all four without reducing scope or delaying anything beyond its current window.

Which set of changes is most appropriate?

- ☐ A Move all four workloads to Amazon EMR on EC2 with Spot Instances to get one consistent platform
- ☐ B Glue with Flex; a Lambda function; SQL inside Redshift; Bedrock batch inference
- ☐ C Glue for all four, right-sized, since a single service is simplest to operate
- ☐ D Athena CTAS for all four, since serverless SQL is the cheapest option available

◆ PAUSE AND DECIDE

Which set of changes is most appropriate?

- A

 Everything on EMR with Spot
A cluster for an hourly nine-megabyte file and for six-theme text classification. Consistency at the cost of fit.
- B

 The right engine for each
No deadline means Flex. Small file means Lambda. Data already in Redshift means SQL. Text classification with no waiting user means batch inference.
- C

 Glue for all four
Simplest to operate and wrong for at least two of them. Glue cannot classify free text at all.
- D

 Athena CTAS for all four
SQL cannot classify survey responses into themes, and the Redshift report would leave the warehouse to come back.

THE SIGNAL

Four workloads, four engines. Standardising on one engine is the tempting wrong answer in this whole module.

Reading an engine question quickly

- ◆ **How big, and is it distributed?** Small and simple means Lambda is live.
- ◆ **Where does the data already sit?** If it is in Redshift, SQL beats a pipeline.
- ◆ **Is a non-Spark engine named?** Hive, HBase, Presto or Trino means EMR.
- ◆ **Is the input unstructured?** No conventional engine reads meaning.
- ◆ **Is anyone waiting?** No deadline unlocks Flex, Spot and batch inference.

ON THE EXAM

Next: orchestration. Step Functions, Amazon MWAA, EventBridge, and what to do when a pipeline fails.